CS3230 : Design and Analysis of Algorithms (Spring 2015)

Tutorial Set #3

[SOLUTION SKETCHES with Suggestions]

[DO NOT give to future students: Let them have a chance to learn.]

R2. Increasing Growth Rates [modified from Spring 2014 Mid-Term Test]

Rank the following functions in *increasing order of growth*; that is, if function $f(n)$ is *immediately* before function $g(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$$g_1(n) = 2^{(2\lg n)} \qquad g_2(n) = 312n^2(\lg\lg n)$$

$$g_3(n) = \sum_{k=1}^{n} \sqrt{n}\cdot(3k+2) \qquad g_4(n) = \sum_{k=1}^{n} 8n(\lg k)$$

Show your steps for the non-trivial cases (such as for g_1, g_3, g_4).

Solution Sketch:

$$g_1(n) << g_2(n) << g_4(n) << g_3(n)$$

$$g_1(n) = 2^{(2\lg n)} = 2^{\lg(n^2)} = n^2 = \Theta(n^2)$$

$$g_2(n) = 312n^2(\lg\lg n) = \Theta(n^2(\lg\lg n))$$

$$g_3(n) = \sum_{k=1}^{n} \sqrt{n}\cdot(3k+2) = n^{0.5}\left(\sum_{k=1}^{n}(3k+2)\right) = \Theta(n^{2.5})$$

$$g_4(n) = \sum_{k=1}^{n} 8n(\lg k) = 8n\sum_{k=1}^{n}(\lg k) = 8n(n\lg n) = \Theta(n^2\lg n)$$

R3. [More practice with Master Theorem] Solve the following recurrence with Master Thm.

(i) $R(n) = 4R(n/16) + 25n^{0.5}(\lg n)^2$

$f(n) = 25n^{0.5}(\lg n)^2 = \Theta(n^{0.5}(\lg^2 n))$, a=4, b=16. So $n^{\log_b a} = n^{\log_{16} 4} = n^{0.5}$.
Since $f(n) = \Theta(n^{0.5}(\lg^2 n)) = \Theta(n^{\log_b a}(\lg^2 n))$. This is **Case 2**. ($k$=2).
Hence, $T(n) = \Theta(n^{0.5}(\lg^3 n))$

(ii) $S(n) = 9S(n/3) + 5n^3$

$f(n) = 5n^3 = \Theta(n^3)$, a=9, b=3. So. $n^{\log_b a} = n^{\log_3 9} = n^2$.
So $f(n) = \Omega(n^{2+\varepsilon})$ for ε=0.5 (for example).
Next check regularity condition, i.e. $a\,f(n/b) \le c\,f(n)$, for some constant $c < 1$.
$a\,f(n/b) = 9*5(n/3)^3 = (\frac{1}{3})*5n^3 \le c\,f(n)$, for $c = (\frac{1}{3}) < 1$.
So the regularity condition is satisfied.
This is **Case 3**. Hence, $T(n) = \Theta(n^3)$

R4. [Very Special Case Quicksort] (motivated by a Question to me after lecture)

Solution Sketch: Worst-case, we have a 1-and-(n–2) split. $T(n) = T(1) + T(n-2) + \Theta(n)$
Since T(1) is a constant, we get $T(n) = T(n-2) + \Theta(n)$. This give $T(n) = \Theta(n^2)$

D1. [Merging Multiple Lists Efficiently]
You are given k sorted arrays, with n elements each, and you want to suitably combine them into a single sorted array (with kn elements).

(a) One strategy to do this is to use a 2-way merge to merge the first two sorted arrays, then merge the result with the third sorted array, then merge in the fourth, and so on. What is the time complexity of this algorithm, in terms of k and n?

Answer: Iterative-merge algorithm: $\Theta(nk^2)$

1st merging step:	Merge array-1 of size n with array-2 of size n,	Time: $2n$
2nd merging step:	Merge result-array of size $2n$ with array-3 of size n,	Time: $3n$
3rd merging step:	Merge result-array of size $3n$ with array-4 of size n,	Time: $4n$
2nd merging step:	Merge result-array of size $4n$ with array-5 of size n,	Time: $5n$
. . .	. . . . .	. . .
last merging step:	Merge result-array of size $(k–1)n$ with array-k of size n,	Time: kn

TOTAL TIME = $n\,(2 + 3 + 4 + \ldots + k) = \Theta(nk^2)$

(b) Design a more *efficient* algorithm for solving this problem. What is the time complexity of your algorithm, in terms of k and n?
Answer: Divide-and-Conquer algorithm: $\Theta(n(k \lg k))$

Top-down view:
Split into 2 halves with $k/2$ arrays each; recursively merge them; then merge their results;
Recurrence: $T(k) = 2T(k/2) + O(nk)$ $T(k) = \Theta(n(k \lg k))$

Bottom-Up view:

Pair up the k arrays (size n) & merge them; get $k/2$ arrays (size $2n$)	Time: $(k/2)*2n$
Pair up the $k/2$ arrays (size $2n$) & merge them; get $k/4$ arrays (size $4n$)	Time: $(k/4)*4n$
Pair up the $k/4$ arrays (size $4n$) & merge them; get $k/8$ arrays (size $8n$)	Time: $(k/8)*8n$
. . . . .	. . .
Pair up the 2 arrays (size $(k/2)n$) & merge them; get 1 array (size kn)	Time: $(1)*kn$

TOTAL TIME = $nk(\lg k) = \Theta(n(k \lg k))$

D4. [Alternative Decision Tree] [1]
In the decision tree (for sorting 3 elements) shown in the lectures (Slide L04.3-Lower Bound on Sorting, Slides 5-6), we compare 1:2 (namely, a_1:a_2) at the root of the tree.

Give an alternative decision tree that compares 1:3 (namely, a_1:a_3) at the root node of the tree and also have *height* 3 (meaning that the worst-case time to sort is still 3 comparisons).

Solution Sketch:
Here, you can "*reuse the method*" (follow the same method as that given in lecture, but starting with compare 1:3 (namely, a_1:a_3) at the root.

Or, you can "*reuse the result*" (start with the solution decision-tree given in lecture that starts with compare 1:2 (namely, a_1:a_2) at the root. Now modify this tree so that we compare 1:3 (namely, a_1:a_3) at the root.

Details for both: DIY.

[1] Solutions to D-problems re-ordered after applying bin packing algorithm to fit in two pages.

D2. (Like Exercise 7.1-1 on p. 171 of [CLRS]) [Understanding PARTITION]
Using the example in lecture notes (L4-Quicksort-Slides-5-16) as a model, illustrate the operation on the array $A[1..12]$ shown below, when you call PARTITION(A, 1, 12).

$A[1..12] = [10, 19, 9, 5, 12, 8, 7, 14, 21, 2, 6, 11]$.

(Use the *first element* as the pivot element.)

Solution sketch:

$A = [10, \mid \mid \mid 19, 9, 5, 12, 8, 7, 14, 21, 2, 6, 11]$.	(select pivot 10)
$A = [10, \mid \mid 19, \mid 9, 5, 12, 8, 7, 14, 21, 2, 6, 11]$.	(19 is big)
$A = [10, \mid 9, \mid 19, \mid 5, 12, 8, 7, 14, 21, 2, 6, 11]$.	(9 is small, swap)
$A = [10, \mid 9, 5, \mid 19, \mid 12, 8, 7, 14, 21, 2, 6, 11]$.	(5 is small, swap)
$A = [10, \mid 9, 5, \mid 19, 12, \mid 8, 7, 14, 21, 2, 6, 11]$.	(12 is big)
$A = [10, \mid 9, 5, 8, \mid 12, 19, \mid 7, 14, 21, 2, 6, 11]$.	(8 is small, swap)
$A = [10, \mid 9, 5, 8, 7, \mid 19, 12, \mid 14, 21, 2, 6, 11]$.	(7 is small, swap)
$A = [10, \mid 9, 5, 8, 7, \mid 19, 12, 14, \mid 21, 2, 6, 11]$.	(14 is big)
$A = [10, \mid 9, 5, 8, 7, \mid 19, 12, 14, 21, \mid 2, 6, 11]$.	(21 is big)
$A = [10, \mid 9, 5, 8, 7, 2, \mid 12, 14, 21, 19, \mid 6, 11]$.	(2 is small, swap)
$A = [10, \mid 9, 5, 8, 7, 2, 6, \mid 14, 21, 19, 12, \mid 11]$.	(6 is small, swap)
$A = [10, \mid 9, 5, 8, 7, 2, 6, \mid 14, 21, 19, 12, 11 \mid]$.	(11 is big)
$A = [6, 9, 5, 8, 7, 2, \mid 10, \mid 14, 21, 19, 12, 11 \mid]$.	(swap pivot to middle)

D3. [Fordy Quicksort, modified from problem by Ken Sung, 2013]
A CS3230 student, called Fordy said that he has an algorithm that, given an unsorted array $A[1..n]$, can find a "*forties pivot*" in time $R(n) = \Theta(6n)$. Using his magic "*forties pivot*", he can guarantee that *each* of the two partitions will have at least X% of the n elements, where X is a value between 40 to 45.

Let $TR(n)$ be the *worst-case performance* of Fordy's Quicksort. Give a recurrence relation for $TR(n)$. Solve for $TR(n)$ and give it in Θ-notations.

Solution sketch:
Worst case cost is when we have a 40% partition (most unbalanced, higher cost), then
$TR(n) = TR(0.40n) + TS(0.60n) + Cn$ (cannot use Master Theorem)) [here C=6]
Draw recursion tree (but can "***re-use the method***" recursion tree)
Each level actually sums up to Cn.
Using the longer (right) branch: $\log_{1.67}(n)$, where $1.67 = (1 / 0.6)$

$$TR(n) \le Cn\log_{1.67}(n) = Cn(\tfrac{\ln 2}{\ln(1.67)}\lg n) = 1.36C(n\lg n)$$

Using the shorter (left) branch: $\log_{2.5}(n)$, where $2.5 = (1 / 0.4)$

$$TR(n) \ge Cn\log_{2.5}(n) = Cn(\tfrac{\ln 2}{\ln 2.5}\lg n) = 0.75C(n\lg n)$$

Hence $TR(n) = \Theta(n \lg n)$.

Note that if you had used a 45% partition, then the lower bound is bigger, while the upper bound is smaller. Hence the 45% partition gives a better solution, and is *not* a worst-case. (As an exercise, you should verify the statement above, namely, carry out this analysis for the 45% partition and check out the lower, upper bounds yourself.)